Assignment 3

STA410, Winter 2026

Assigned on March 20, 2026.
Submit by 11:59 pm on April 3, 2026 on Qeurcus.

1. **(Randomized SVD)** This problem will construct a matrix decomposition of $X \in \mathbb{R}^{n\times p}$ known as the singular value decomposition (SVD):

$$X = UDV^T, \qquad U \in \mathbb{R}^{n\times r}, D \in \mathbb{R}^{r\times r}, V \in \mathbb{R}^{p\times r},$$

where $r \leq \min(n, p)$. The *randomized SVD* is an efficient algorithm to find the decomposition when n and p are very large but $r \ll p$. This algorithm works as follows:

1. A random matrix $P \in \mathbb{R}^{p\times r}$ is generated;
2. The matrix $Z \in \mathbb{R}^{n\times r}$ is computed as $Z = XP$. This step "randomly samples" the column space of X using r samples in the columns of P, which lead to weighted combinations of the columns of X in the columns of Z;
3. A QR decomposition $Z = QR$ is computed where $Q \in \mathbb{R}^{n\times r}$ is a matrix of orthogonal columns and $R \in \mathbb{R}^{r\times r}$. Q defines an orthogonal basis for Z which approximates the column space of X;
4. X is now projected onto the orthogonal directions Q as $Y = Q^T X$ where $Y \in \mathbb{R}^{r\times p}$;
5. The SVD of $Y = U_Y D_Y V_Y^T$ is computed where $U_Y \in \mathbb{R}^{r\times r}, D_Y \in \mathbb{R}^{r\times r}, V_Y \in \mathbb{R}^{p\times r}$.
6. The original matrix is approximated as $X \approx \widetilde{X} = QU_Y D_Y V_Y^T$.

In summary, this approach projects X into a lower dimensional space using Q^T, performs the computations in that lower dimensional space (where they're less expensive), and then projects the result back out to the original space with Q. Overall, this required computations based on Z and Y instead of X, which is more efficient when $r \ll p$

The following questions will implement and apply randomized SVD to analyze the `mtcars` dataset, which contains fuel consumption and 10 design/performance aspects for 32 automobiles. You can load this dataset in Python using `X = sm.datasets.get_rdataset("mtcars").data.values`.

(a) Center and scale the columns of X so that they have mean 0 and standard deviation 1. (Hint: Center and scale each column independently, i.e., subtract column means and divide by column standard deviations)

(b) Describe how centering and scaling affects the condition number, and show how this relates to the spread of singular values. (Hints: The following functions may be useful `np.linalg.cond`, `np.linalg.qr`, `np.linalg.svd`; We also recommend plotting the singular values on a log-axis). Describe the benefit of centering in terms of the condition number and its relation to the singular values.

(c) Generate a random $P \in \mathbb{R}^{p\times r}$ matrix where $r = 5$ with independent standard Gaussian entries. Given P, implement steps 2-6 of the randomized SVD procedure above. (Hint: Be mindful of the `full_matrices=True/False` argument of the `svd` function so that the dimension of the returned matrices is as prescribed by the randomized SVD procedure.)

(d) Which of the following is the smallest ratio of r to p for which

$$\|X - QU_Y D_Y V_Y^T\|_1 / \|X\|_1 < 0.2,$$

where $\|A\|_1$ is the sum of the absolute values of matrix A. Try increasing r starting from 1.

2. (**Iterative Optimization Methods**) In this problem we consider minimizing the function

$$f(x) = x^2 - 2\alpha x + \lambda |x|^\gamma,$$

where $\lambda > 0$ and $0 < \gamma < 1$. This problem arises, in a somewhat more complicated form, in sparse regression problems. The function $|x|^\gamma$ has a "cusp" at 0, which means that if λ is sufficiently large then f is minimized at $x = 0$.

(a) Show that f is minimized at $x = 0$ if, and only if,

$$\lambda \geq \frac{2}{2-\gamma}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma} |\alpha|^{2-\gamma}.$$

Otherwise, f is minimized at x^* satisfying $f'(x^*) = 0$.

(b) Using Python, compare the following two iterative algorithms for computing x^* (when the above condition does not hold):

i. Set $x_0 = \alpha$ and define

$$x_k = \alpha - \frac{\lambda\gamma}{2} \cdot \frac{|x_{k-1}|^\gamma}{x_{k-1}}, \quad k = 1, 2, 3, \ldots$$

ii. The Newton–Raphson algorithm with $x_0 = \alpha$.

Use different values of α, γ, and λ to test these algorithms. Which algorithm is faster?

(c) Functions like f arise in so-called *bridge estimation* in linear regression, which are generalizations of the LASSO. Such estimation combines the features of ridge regression (which shrinks least squares estimates toward 0) and model selection methods (which produce exact 0 estimates for some or all parameters). Given $\gamma > 0$ and $\lambda > 0$), bridge estimates minimize the following objective for coefficients $\beta \in \mathbb{R}^d$ given data $y^i \in \mathbb{R}$ and $x^i \in \mathbb{R}^d$ for $i = 1, \ldots, N$:

$$\sum_{i=1}^{N} (y^i - (x^i)^T\beta)^2 + \lambda \sum_{j=1}^{d} |\beta_j|^\gamma.$$

Describe how the algorithms in part (b) could be used to define a coordinate descent algorithm to find β minimizing the above expression iteratively one parameter at a time.

3. (**Gradient Descent with Step-Size**) The Rosenbrock function is a classical test problem for optimization algorithms. It is defined as

$$f(x) = (1 - x_1)^2 + (x_2 - x_1^2)^2.$$

This function is often referred to as the *banana function* due to its characteristic curved valley shape. The function has a global minimum at $(x_1^*, x_2^*) = (1, 1)$, with $f(x_1^*, x_2^*) = 0$. Although the valley of the function is easy to locate, it is very flat near the minimum, making it challenging for optimization algorithms to converge efficiently.

(a) Compute the gradient and Hessian of the Rosenbrock function. Implement a function that, given x, returns: 1) the function value $f(x)$, 2) the gradient $\nabla f(x)$ as a 2×1 vector, and 3) the Hessian $\nabla^2 f(x)$ as a 2×2 matrix.

(b) We now consider iterative optimization methods starting from an initial point x_0. At iteration k, gradient descent updates the iterate according to

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

where $\alpha_k > 0$ is the step size. The iteration is terminated when $\|\nabla f(x_k)\| < \varepsilon$, or when a maximum number of iterations $N_{\max}$ is reached. Implement gradient descent with a fixed step size α. Your algorithm should: 1) take as input x_0, α, ε, and $N_{\max}$, 2) return the optimization path $\{x_k\}$, and 3) return the convergence history $\{f(x_k)\}$.

Choose α such that the method converges in fewer than 300 iterations and answer the following: 1) How many iterations are required to reach the lowest contour of the Rosenbrock valley? 2) How many additional iterations are required to converge near the minimum? 3) Briefly explain why convergence slows down in this region.

(c) In this part, we will improve the gradient descent method by selecting the step size α_k using a bisection line search. Let the descent direction be

$$u_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

Define the one-dimensional function

$$\phi(\alpha) = f(x_k + \alpha u_k).$$

To determine α_k, solve $\phi'(\alpha) = 0$ we can use a bisection method over the interval $[0, \alpha_{\text{UB}}]$ for some appropriate maximum value α_{UB}.

Implement gradient descent using this adaptive step size strategy. Compare the fixed step size and bisection line search methods and answer the following: 1) How do the optimization paths differ? 2) Which method converges faster? 3) What additional computational cost is introduced by the bisection search?

4. **(EM Algorithm for a Mixture of Exponentials)** Let $x^1, \ldots, x^N$ be independent observations from a two-component mixture of exponential distributions:

$$p(x;\, \lambda_1, \lambda_2, p) = \theta \cdot \lambda_1 e^{-\lambda_1 x} + (1-\theta) \cdot \lambda_2 e^{-\lambda_2 x}, \qquad x \geq 0,$$

where $\lambda_1, \lambda_2 > 0$ and $0 < \theta < 1$ are unknown parameters.

(a) Define appropriate complete data $Y^i = (X^i, \Delta_i)$ and write down the complete data log-likelihood $\ln L_c(\lambda_1, \lambda_2, \theta)$.

(b) Derive the E-step in EM by computing $\hat{\delta}_i(\lambda_1, \lambda_2, \theta) = \mathbb{E}\big[\Delta_i \mid X^i = x^i\big]$ in closed form.

(c) Derive the M-step update equations for $\hat{\lambda}_1$, $\hat{\lambda}_2$, and $\hat{\theta}$ in closed form.

(d) Implement the EM algorithm in Python. Generate $N = 200$ observations using true parameters $\lambda_1 = 1$, $\lambda_2 = 5$, $\theta = 0.4$, and run the algorithm to convergence. Report your estimates.

(e) Plot the log-likelihood at each iteration to illustrate the monotone increasing property of the EM algorithm. How many iterations are required until convergence?

(f) The observed data log-likelihood is:

$$\ln L(\lambda_1, \lambda_2, \theta) = \sum_{i=1}^{n} \ln\Big(\theta\, \lambda_1 e^{-\lambda_1 x^i} + (1-\theta)\, \lambda_2 e^{-\lambda_2 x^i}\Big).$$

Is this function unimodal? Try at least three different starting values and comment on whether your EM runs converge to the same solution.